Estructura de datos

Árboles binarios de búsqueda

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2026-II

UACM
Universidad Autónoma de la Ciudad de México
NADA HUMANO ME ES AJENO

Índice

1 Introducción a los árboles

2 Árboles binarios

3 Recorridos de árboles

4 Arboles binarios de búsqueda

5 Implementación de un ABB

6 Búsqueda

7 Inserción

8 Eliminación

9 Máximos y mínimos

10 Recorridos

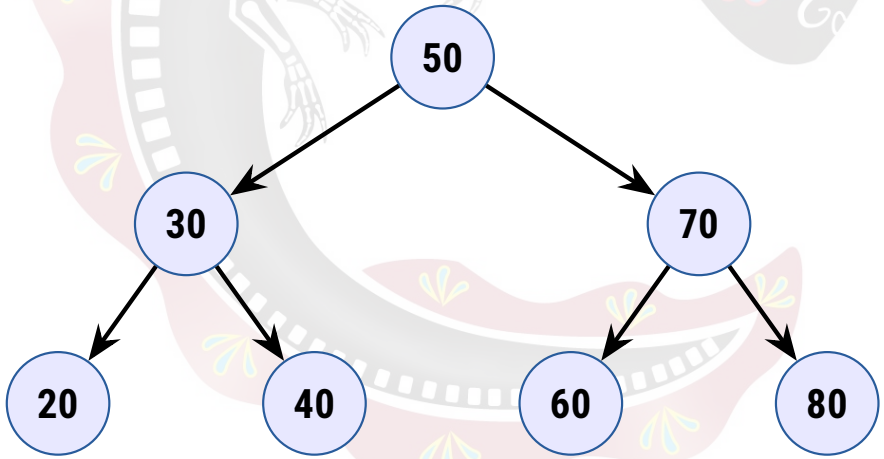
11 Altura

¿Qué es un árbol?

Definición

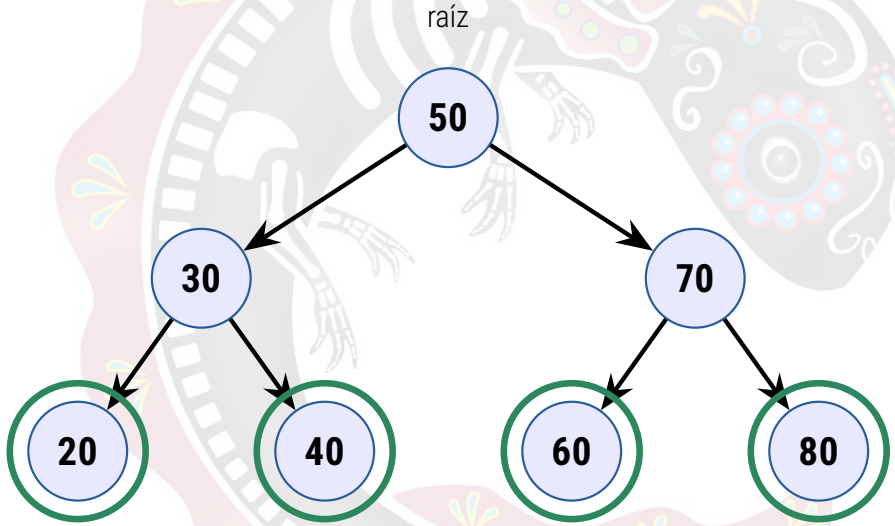
Estructura de datos **jerárquica** formada por nodos y enlaces, con una **raíz** y sin ciclos. Todo nodo **distinto de la raíz** tiene exactamente **un padre.**

- Representa relaciones **padre–hijo.**
- Cada nodo puede ser **raíz** de un subárbol.
- **No existe** un camino alternativo entre **dos nodos.**

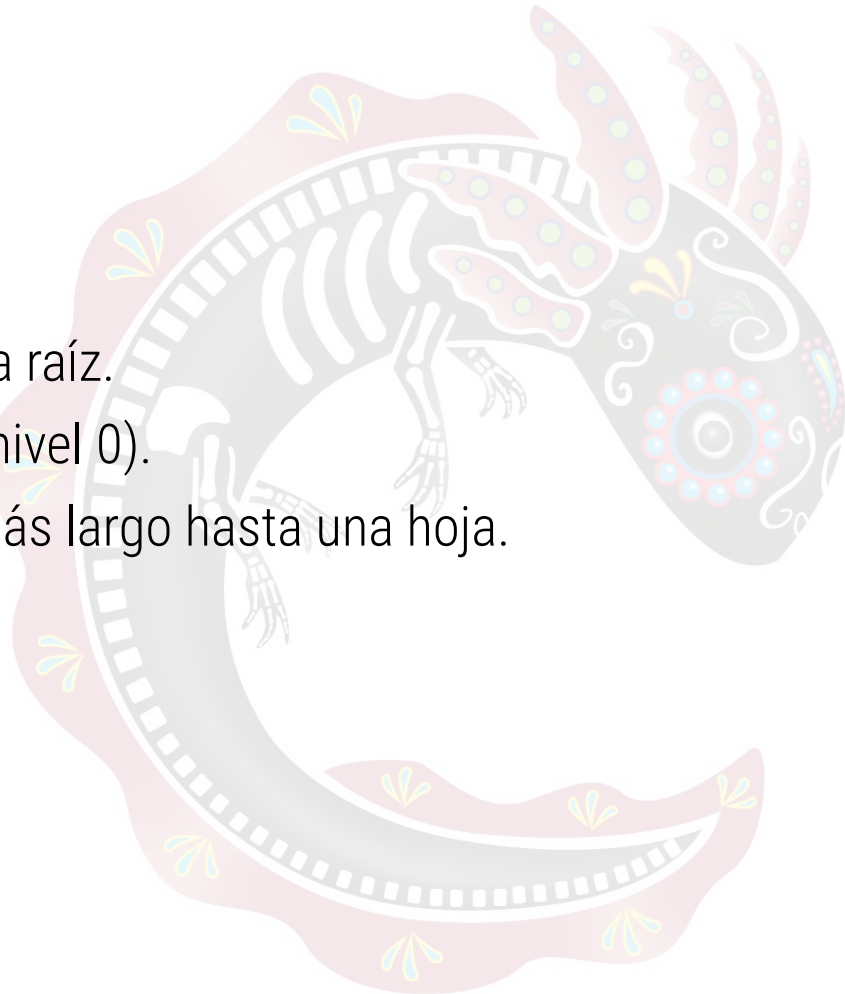


Terminología: raíz, padre, hijos y hojas

- **Raíz:** 50.
- **Padre de 30 y 70:** 50.
- **Hijos de 30:** 20 y 40.
- **Hermanos:** 20 y 40.
- **Hojas:** 20, 40, 60 y 80.
- **Nodos internos:** 50, 30 y 70.

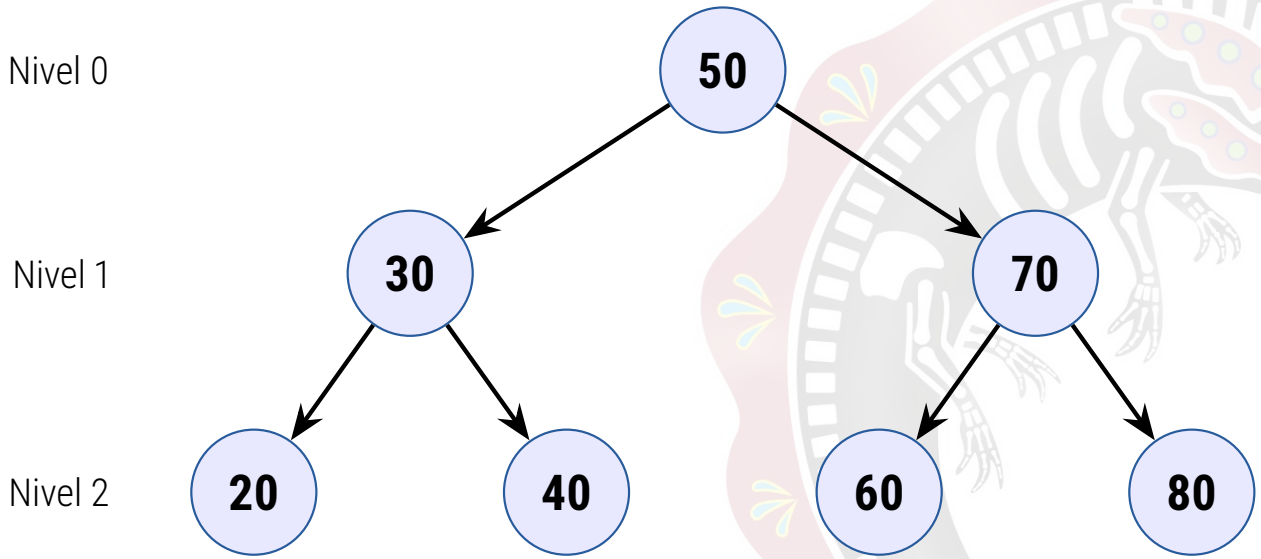


Terminología: nivel, profundidad, altura y grado



- **Profundidad:** número de aristas desde la raíz.
- **Nivel:** profundidad (convención: raíz en nivel 0).
- **Altura de un nodo:** aristas del camino más largo hasta una hoja.
- **Altura del árbol:** altura de la raíz.
- **Grado de un nodo:** cantidad de hijos.

Terminología: nivel, profundidad, altura y grado



Convención de esta implementación

Árbol vacío: altura -1; hoja: altura 0.

Propiedades de un árbol

Para un **árbol no vacío** con n nodos:

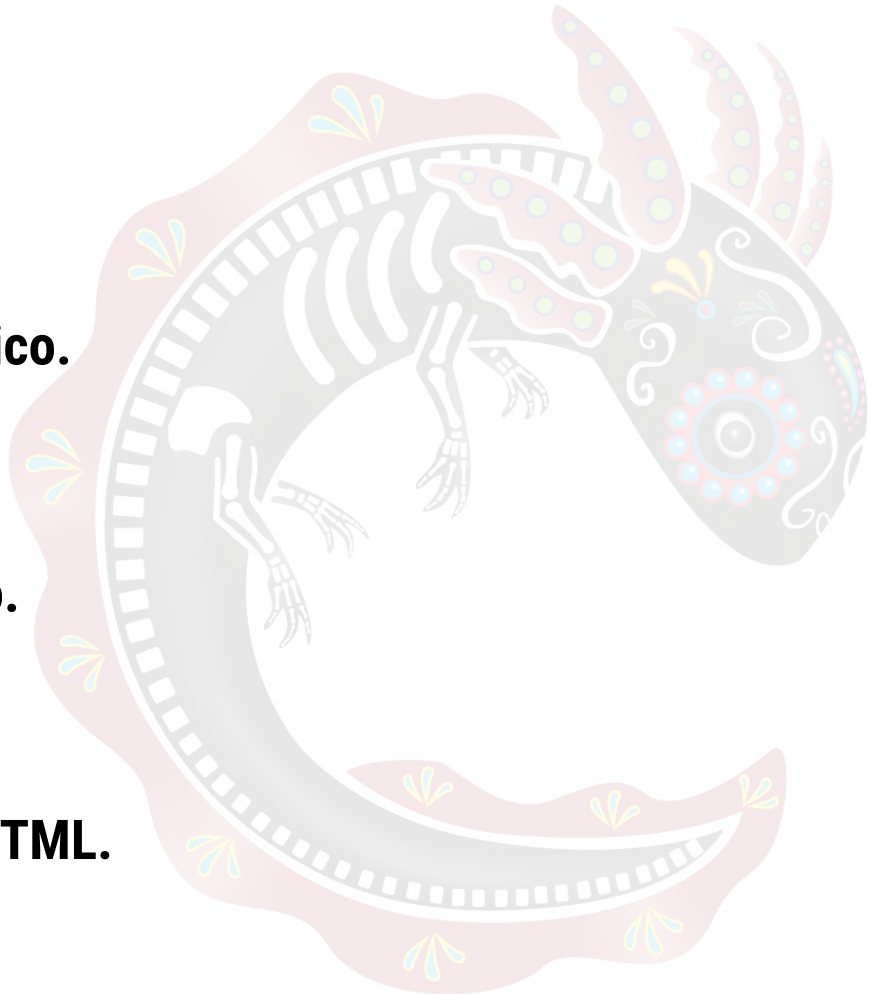
- Tiene **exactamente** $n - 1$ aristas.
- Existe **un único camino** entre la raíz y cualquier nodo.
- **Cada subárbol** es también **un árbol.**
- La **cantidad de hijos** de un nodo depende del **tipo de árbol.**

Ejemplo

El árbol mostrado tiene $n = 7$ nodos y 6 aristas. Su altura es 2 (medida en aristas).

Aplicaciones de los árboles

- **Jerarquías de archivos** y directorios.
- Árboles de expresión y **análisis sintáctico.**
- Índices de **bases de datos.**
- **Árboles de decisión.**
- Búsqueda y **almacenamiento ordenado.**
- **Compresión de datos** (Huffman).
- **Árboles de ejecución** y llamadas.
- Representación de documentos **XML/HTML.**

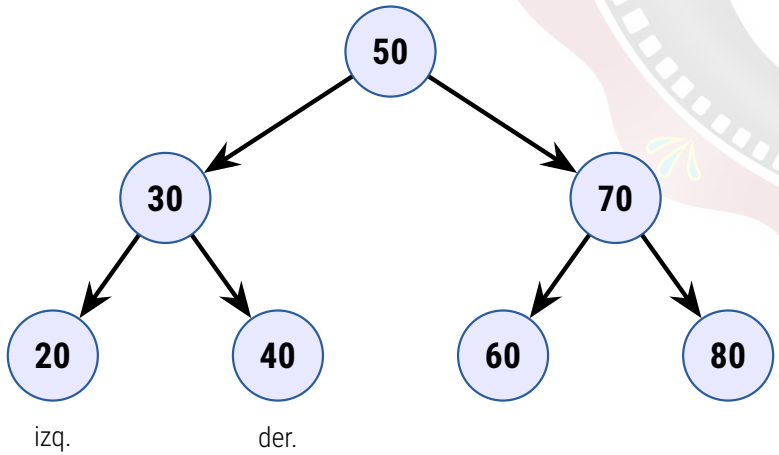


Definición de árbol binario

Definición

Un **árbol binario** es un árbol en el que cada nodo tiene **a lo sumo dos hijos,** identificados como **izquierdo** y **derecho**.

- Ambos hijos **pueden** estar ausentes.
- Puede existir **solo uno** de los dos.
- **La posición** izquierda/derecha **importa.**
- **No exige** que los elementos estén ordenados.



Propiedades de los árboles binarios

Con la raíz en el nivel 0 y altura h:

- **Máximo de nodos** en el nivel k: 2^k.
- **Máximo de nodos** en un árbol de altura h: $2^{h+1} - 1$.
- **Altura mínima** para $n > 0$ nodos: $\lceil \log_2(n+1) \rceil - 1$.
- **Altura máxima** para $n > 0$ nodos: $n - 1$ (árbol degenerado).

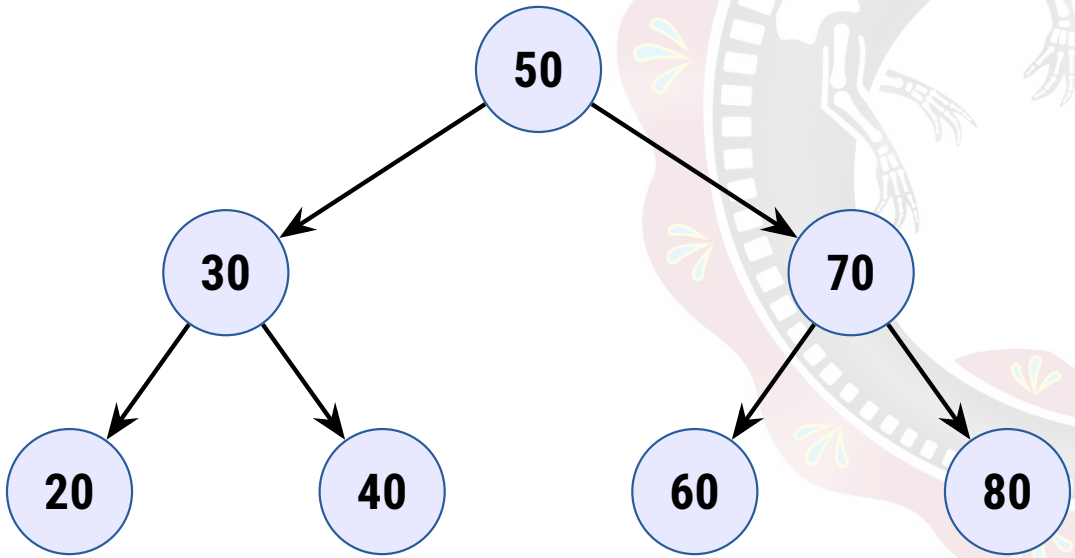
Ejemplo

Con altura $h = 2$ puede haber como máximo $2^3 - 1 = 7$ nodos.

Formas frecuentes de árboles binarios

Definición

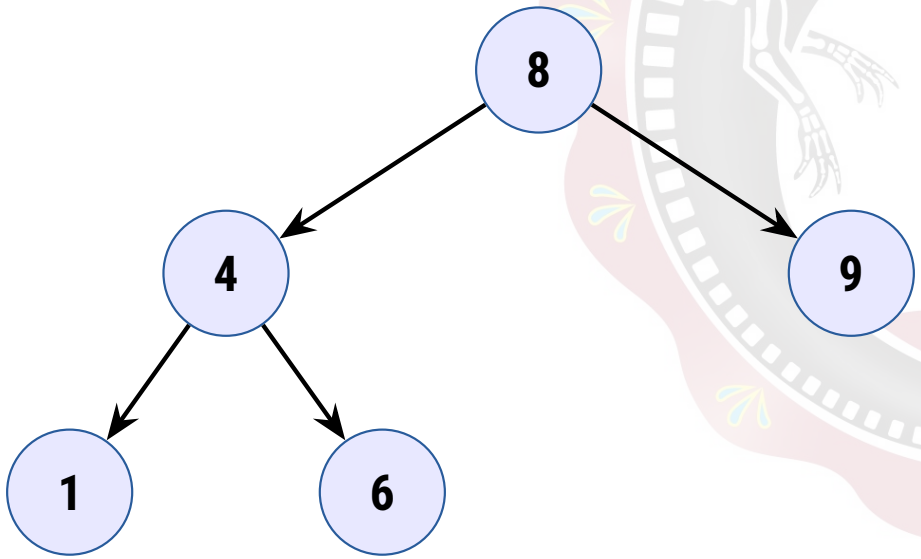
Un arbol **perfecto** es aquel en el que **todos los niveles** están completos.



Formas frecuentes de árboles binarios

Definición

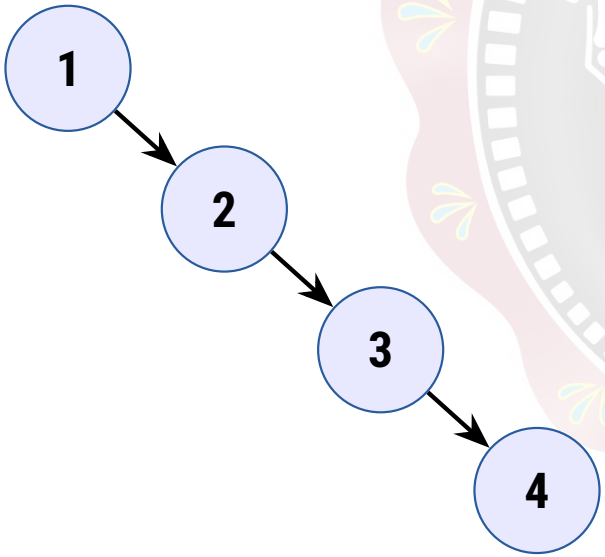
Un árbol **completo** es aquel en el que todos los niveles están llenos, **excepto** posiblemente el **último,** el cual se rellena de **izquierda a derecha.**



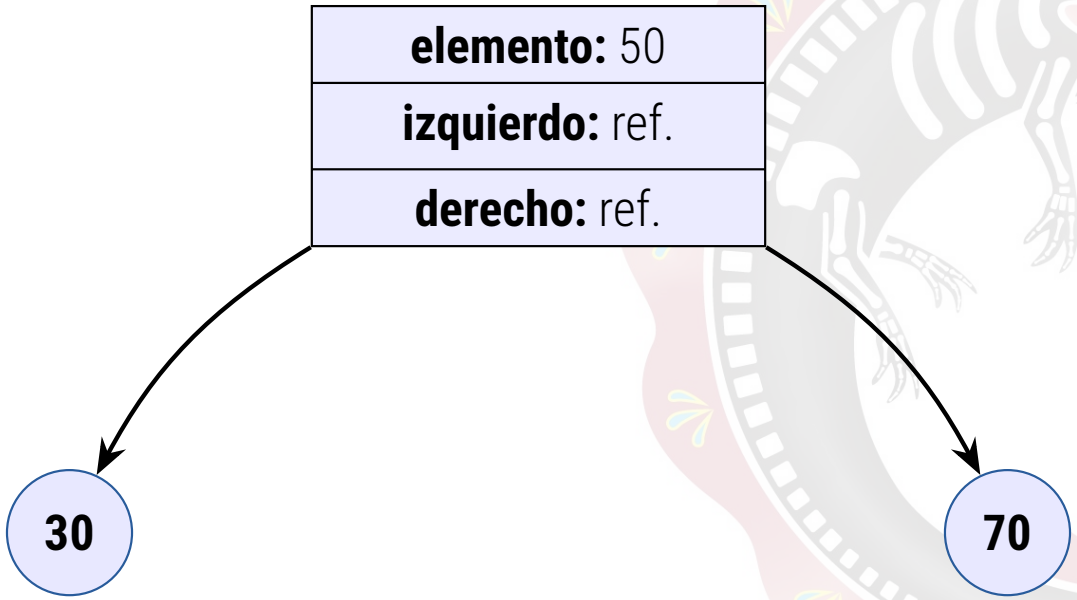
Formas frecuentes de árboles binarios

Definición

Un árbol **degenerado** es aquel en el que cada nodo interno tiene **exactamente 1 hijo.**



Representación enlazada: `NodoArbol<E>`



Representación enlazada: NodoArbol<E>

```
public class NodoArbol<E> {
  private E elemento;
  private NodoArbol<E> izquierdo;
  private NodoArbol<E> derecho;

  public NodoArbol(E elemento) {
    this.elemento = elemento;
    izquierdo = null;
    derecho = null;
  }
  // Getters y setters
}
```

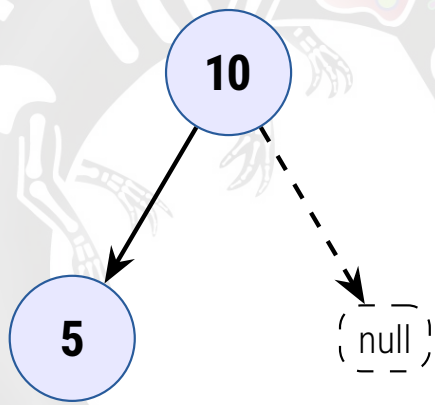
Referencias y casos base

Referencias

`izquierdo` y `derecho` contienen **referencias** a otros nodos o `null`.

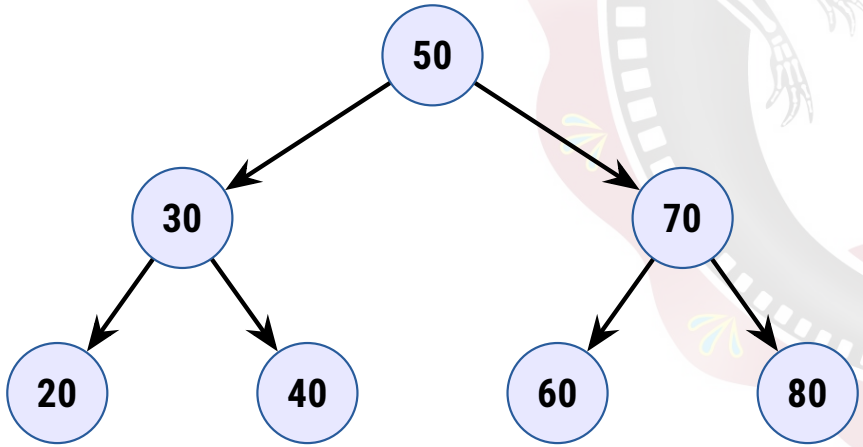
Recursividad

`actual == null` es el **caso base** de la mayoría de los algoritmos sobre árboles.



Recorridos: visitar todos los nodos

- **Preorden (N-I-D):** nodo, izquierdo, derecho.
- **Inorden (I-N-D):** izquierdo, nodo, derecho.
- **Postorden (I-D-N):** izquierdo, derecho, nodo.
- **Por niveles:** amplitud, de arriba hacia abajo y de izquierda a derecha.



Preorden: nodo – izquierdo – derecho

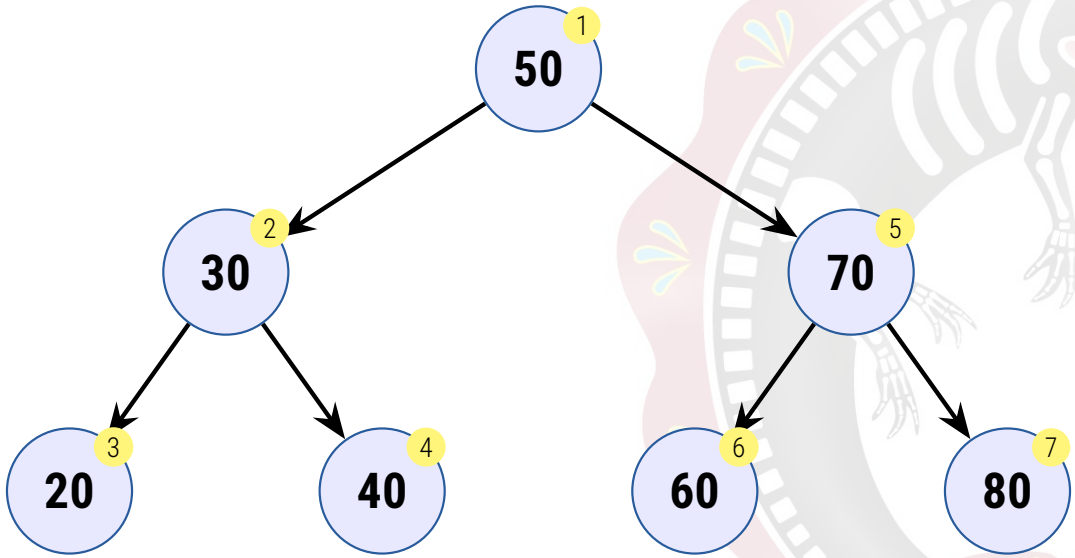
Pseudocódigo

Entrada: $nodo$ inicial para el recorrido, Lista L

```
1: procedure PREORDEN(nodo, L)
2:     if nodo = null then
3:         return
4:     end if
5:     L.insertar(nodo.elemento)
6:     PREORDEN(nodo.izquierdo, L)
7:     PREORDEN(nodo.derecho, L)
8: end procedure
```



Preorden: nodo – izquierdo – derecho



Resultado: 50, 30, 20, 40, 70, 60, 80.

Inorden: izquierdo – nodo – derecho

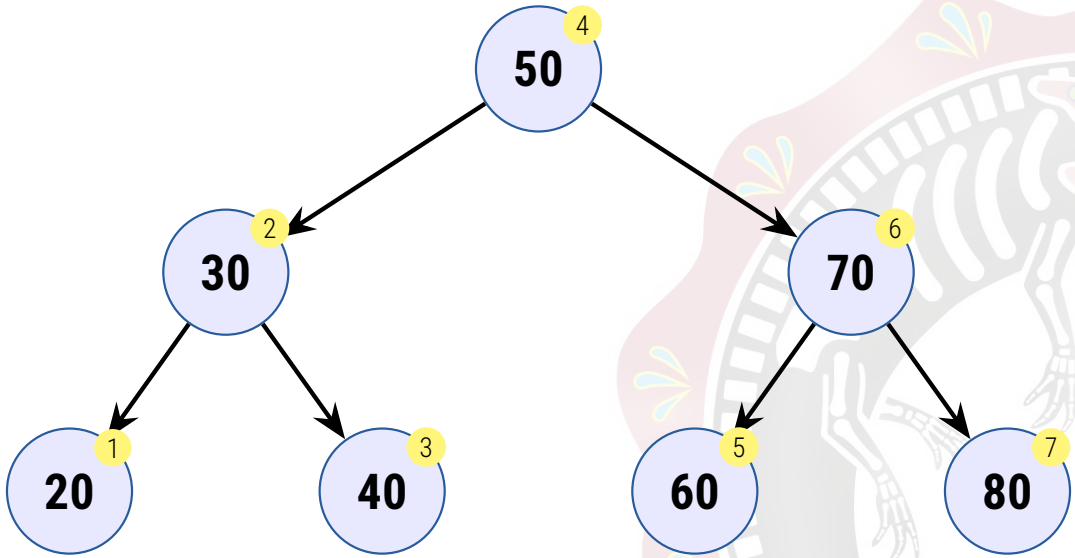
Pseudocódigo

Entrada: $nodo$ inicial para el recorrido, Lista L

```
1: procedure INORDEN(nodo, L)
2:     if nodo = null then
3:         return
4:     end if
5:     INORDEN(nodo.izquierdo, L)
6:     L.insertar(nodo.elemento)
7:     INORDEN(nodo.derecho, L)
8: end procedure
```



Inorden: izquierdo – nodo – derecho



Resultado: 20, 30, 40, 50, 60, 70, 80.

¡Importante!

En un árbol binario de búsqueda, el **recorrido inorden** entrega los elementos en **orden ascendente.**

Postorden: izquierdo – derecho – nodo

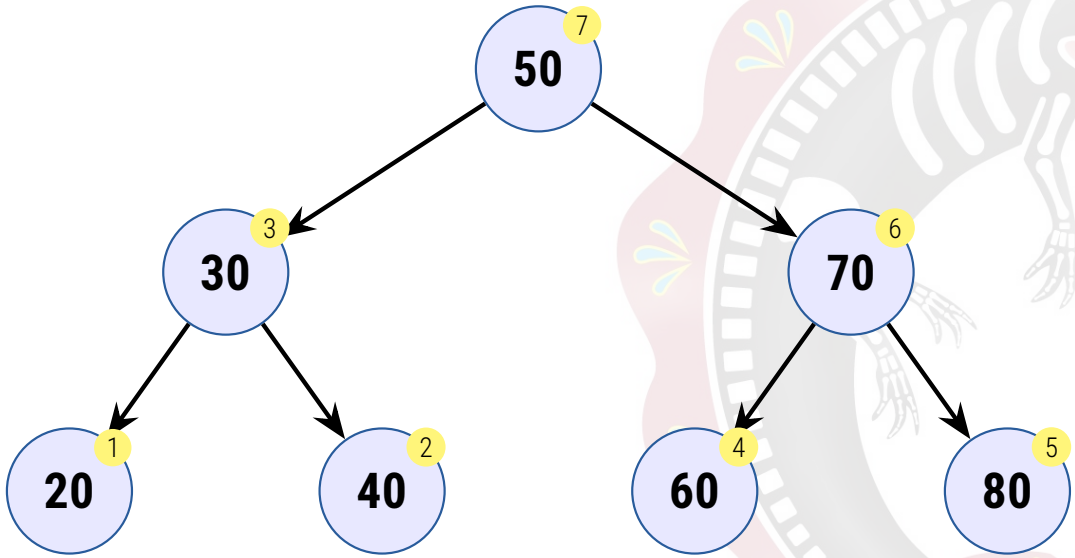
Pseudocódigo

Entrada: $nodo$ inicial para el recorrido, Lista L

```
1: procedure POSTORDEN(nodo, L)
2:     if nodo = null then
3:         return
4:     end if
5:     POSTORDEN(nodo.izquierdo, L)
6:     POSTORDEN(nodo.derecho, L)
7:     L.insertar(nodo.elemento)
8: end procedure
```



Postorden: izquierdo – derecho – nodo



Resultado: 20, 40, 30, 60, 80, 70, 50.

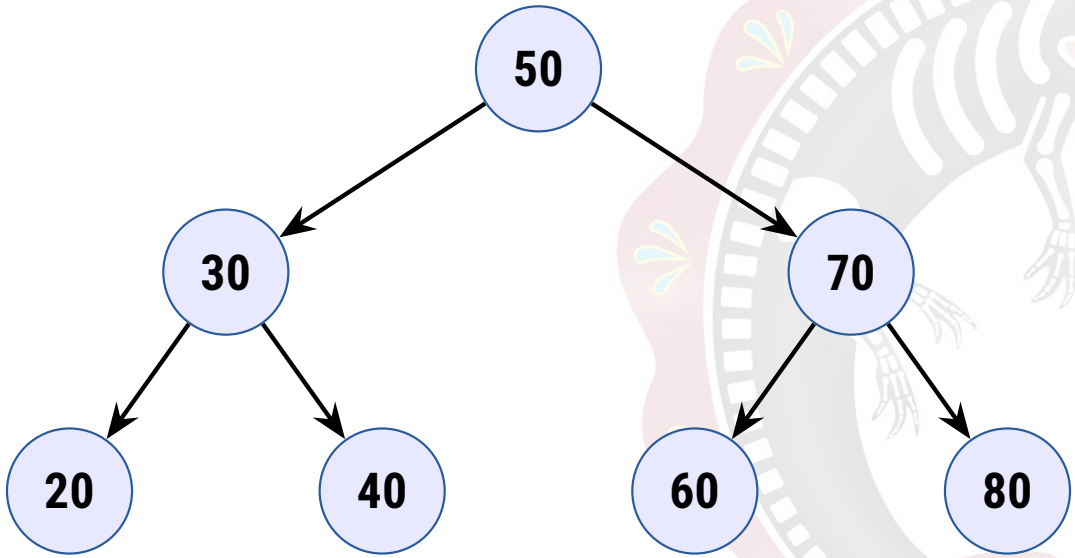
Por niveles: recorrido en amplitud (BFS)

Entrada: Nodo $raiz$ del árbol

1: $lista \leftarrow Lista()$
2: **if** $raiz = null$ **then**
3: **return** L
4: **end if**
5: $cola \leftarrow Cola()$
6: $cola.encolar(raiz)$
7: **while not** $cola.estaVacia()$ **do**
8: $actual \leftarrow cola.desencolar()$
9: $lista.insertar(actual.elemento)$
10: **if** $actual.izquierdo \neq null$ **then**
11: $cola.encolar(actual.izquierdo)$
12: **end if**
13: **if** $actual.derecho \neq null$ **then**
14: $cola.encolar(actual.derecho)$
15: **end if**
16: **end while**



Por niveles: recorrido en amplitud (BFS)



Resultado: 50, 30, 70, 20, 40, 60, 80

Evolución de la cola durante BFS

Se extrae	Se encolan	Cola resultante (frente → final)
–	50	[50]
50	30, 70	[30, 70]
30	20, 40	[70, 20, 40]
70	60, 80	[20, 40, 60, 80]
20	–	[40, 60, 80]
40	–	[60, 80]
60	–	[80]
80	–	[ ]

Reutilización de TDA

La cola almacena `NodoArbol<E>`, mientras la lista de resultados almacena elementos `E`.

Comparación de recorridos

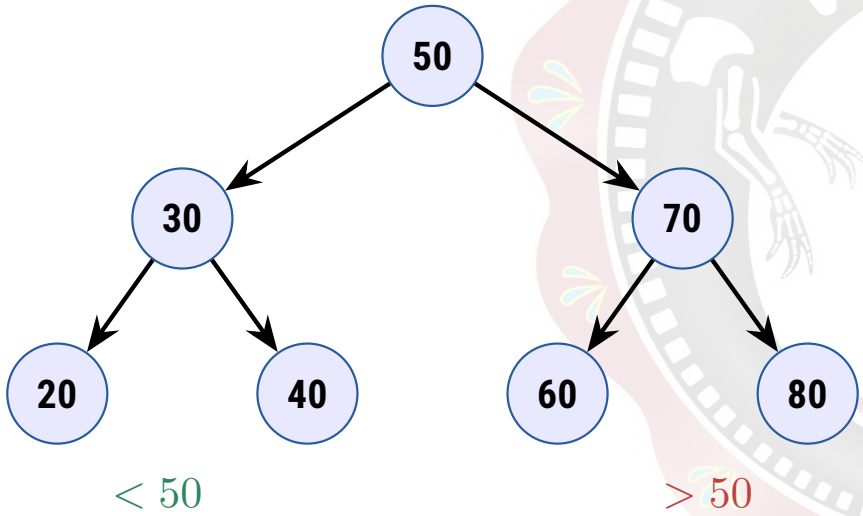
Recorrido	Regla	Secuencia
Preorden	N-I-D	50, 30, 20, 40, 70, 60, 80
Inorden	I-N-D	20, 30, 40, 50, 60, 70, 80
Postorden	I-D-N	20, 40, 30, 60, 80, 70, 50
Por niveles	BFS	50, 30, 70, 20, 40, 60, 80

- Los recorridos **visitan** n nodos: tiempo $O(n)$.
- La recursión utiliza espacio **proporcional a la altura** h, BFS utiliza una cola cuyo tamaño máximo es **proporcional al ancho del árbol.**
- Al devolver `Lista<E>`, se almacenan n **elementos adicionales:** $O(n)$.

Árbol binario de búsqueda (ABB)

Para cada nodo con elemento x:

- Todo elemento del **subárbol izquierdo** es **menor** que x.
- Todo elemento del **subárbol derecho** es **mayor** que x.
- Ambos subárboles **cumplen recursivamente** la misma propiedad.



Política adoptada

No se permiten duplicados ni valores `null`. Los elementos **deben poder compararse** mediante `compareTo`.

Operaciones del TDA ArbolBusqueda

Algunas de las **operaciones definidas** para el TDA son:

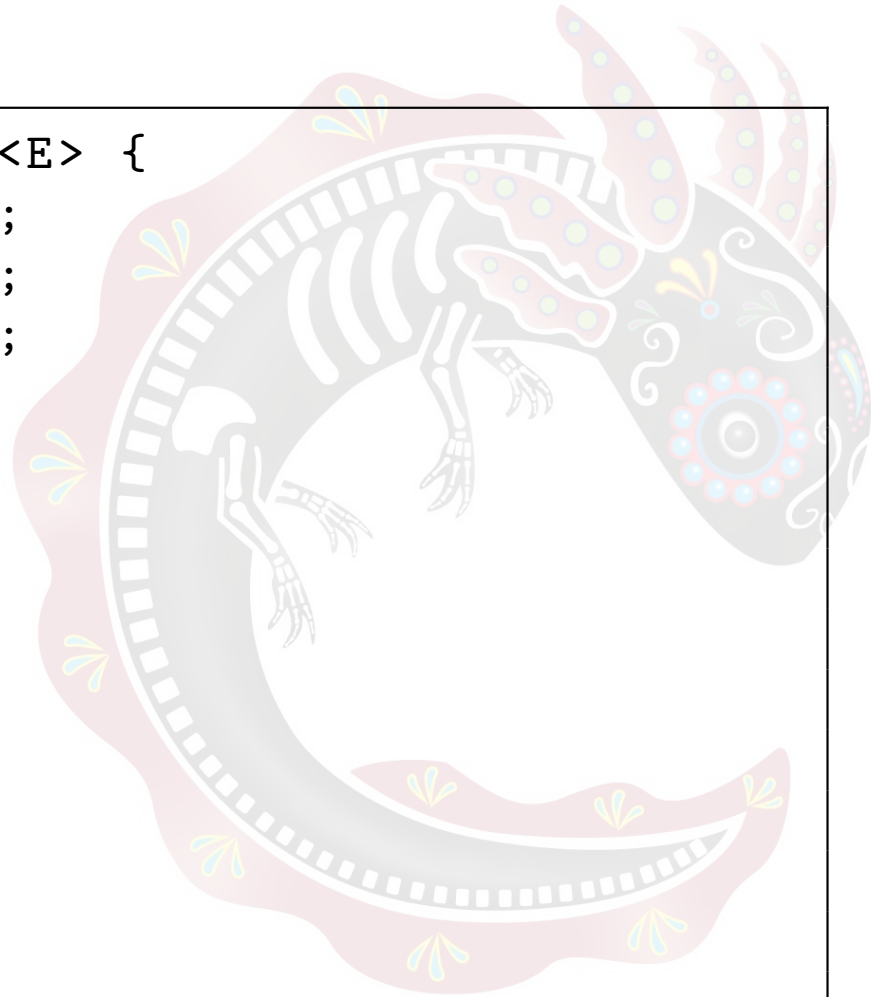
`insertar(E)`	**Agrega** un elemento al árbol
`eliminar(E)`	**Elimina y devuelve** el elemento.
`contiene(E)`	**Consulta** si el elemento está dentro del árbol.
`minimo()`	**Devuelve** el elemento mínimo.
`maximo()`	**Devuelve** el elemento máximo.
`tamanio()`	**Devuelve** el número de elementos almacenados.
`estaVacia()`	Determina si el árbol **contiene elementos.**
`limpiar()`	**Elimina todos** los elementos.
`altura()`	**Devuelve** la altura del árbol.

Abstracción

El TDA establece qué operaciones pueden realizarse, pero no determina **cómo deben almacenarse** internamente los elementos.

Interfaz `ArbolBusqueda<E>` en Java

```
public interface ArbolBusqueda<E> {
  boolean insertar(E elemento);
  boolean eliminar(E elemento);
  boolean contiene(E elemento);
  E minimo();
  E maximo();
  int tamanio();
  boolean estaVacio();
  void limpiar();
  int altura();
  Lista<E> preorden();
  Lista<E> inorden();
  Lista<E> postorden();
  Lista<E> porNiveles();
}
```



Interfaz e implementación

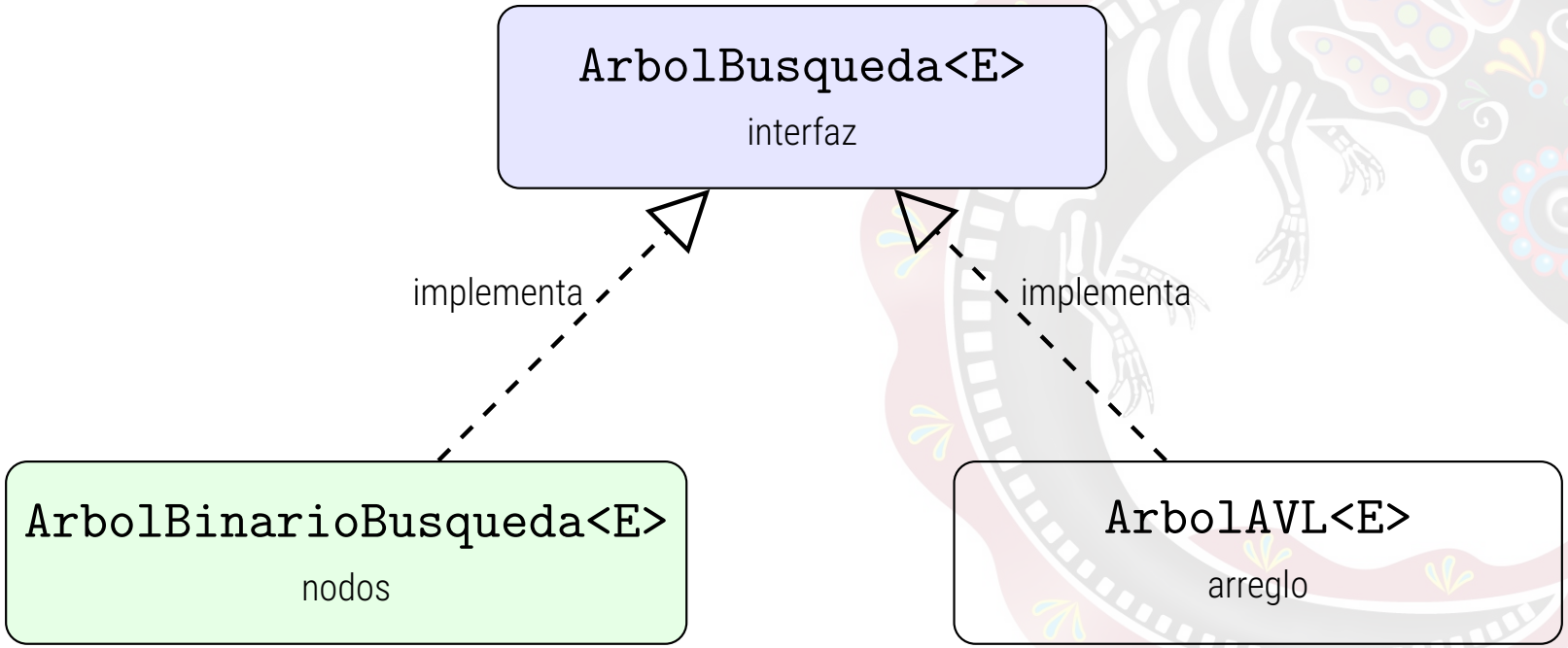
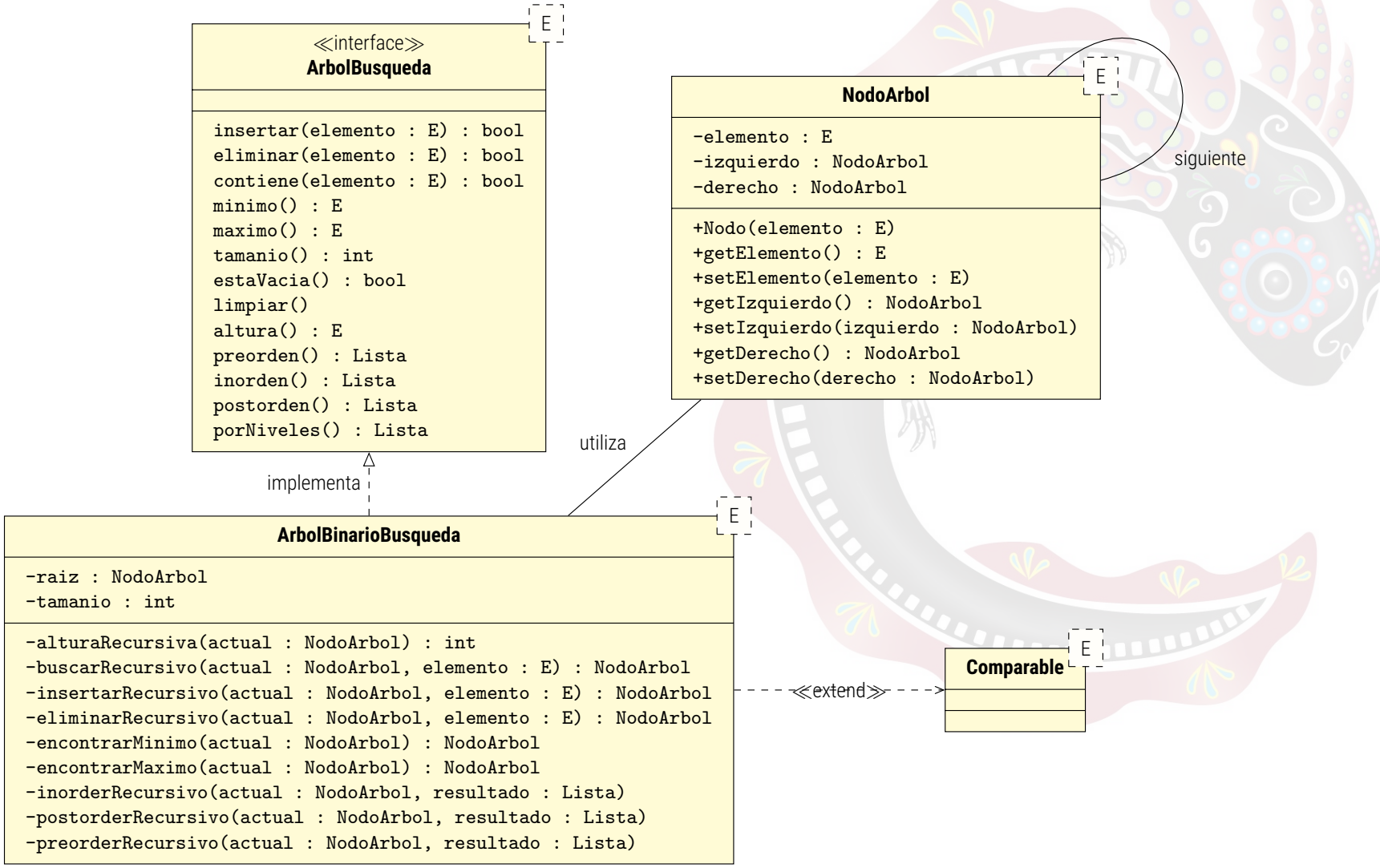


Diagrama de clases



Estructura de `ArbolBinarioBusqueda<E>`

```
public class ArbolBinarioBusqueda<E extends Comparable<E
    >>
implements ArbolBusqueda<E> {

  private NodoArbol<E> raiz;
  private int tamanio;

  public ArbolBinarioBusqueda() {
    raiz = null;
    tamanio = 0;
  }
}
```

Comparación

El método `compareTo` **determina** si un elemento debe buscarse o insertarse a la **izquierda,** a la **derecha** o si **ya existe.**

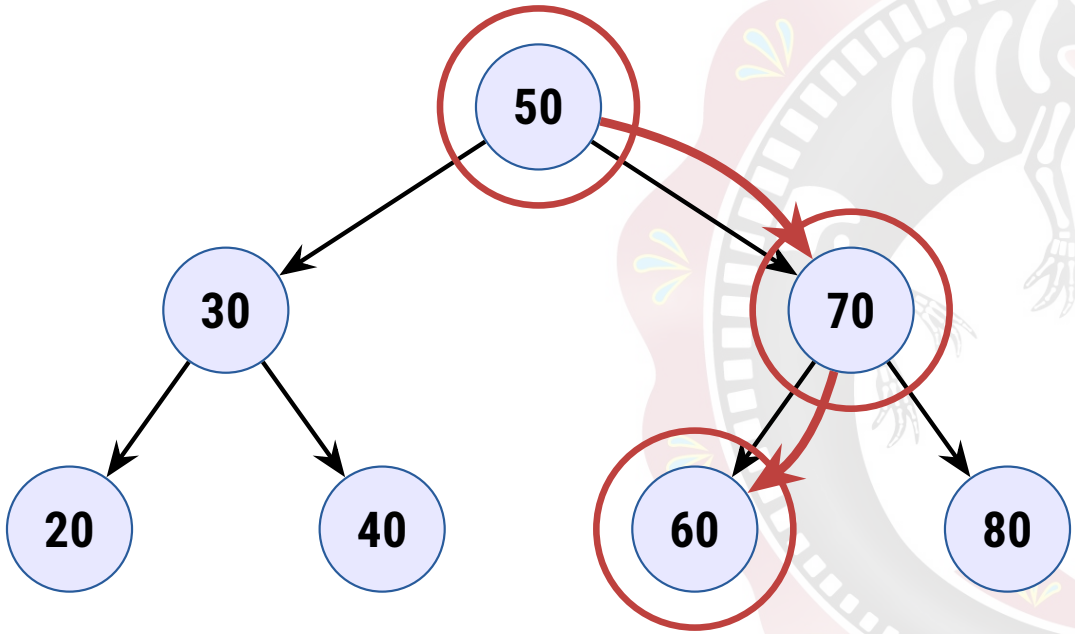
Búsqueda: elegir un único subárbol

Pseudocódigo

```
1:  procedure BUSCAR(actual, valor)
2:      if nodo = null then
3:          return
4:      end if
5:      if valor = actual.elemento then
6:          return actual
7:      end if
8:      if valor < actual.elemento then
9:          return BUSCAR(actual.izquierdo, valor)
10:     end if
11:     return BUSCAR(actual.derecho, valor)
12: end procedure
```



Búsqueda: elegir un único subárbol



Ejemplo: buscar 60: $50 \to 70 \to 60$.

Método `contiene(E elemento)`



Pseudocódigo

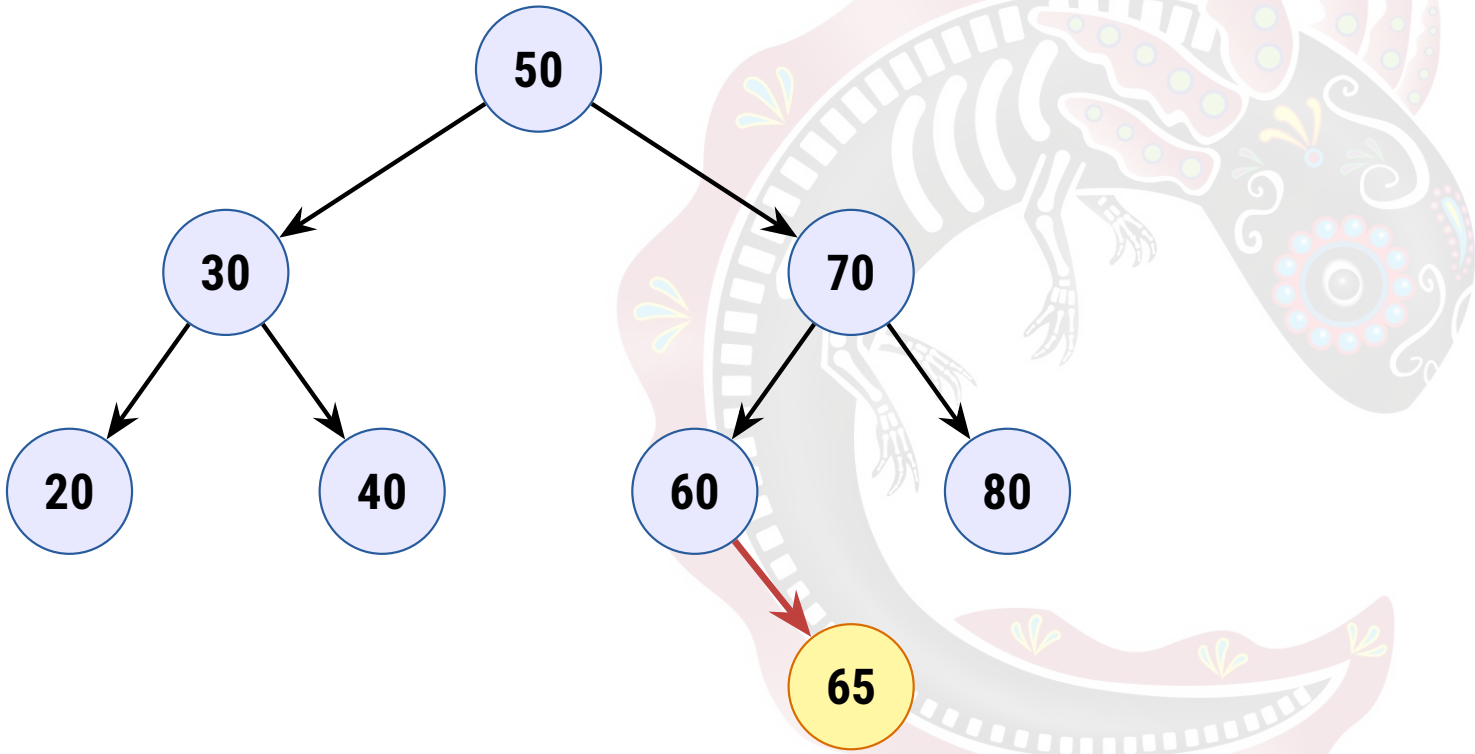
1: **procedure** CONTIENE($elemento$)
2: **if** $elemento = null$ **then**
3: **return** $false$
4: **end if**
5: $encontrado \leftarrow$ BUSCARRECURSIVO($raiz, elemento$)
6: **return** $encontrado \neq null$
7: **end procedure**

Inserción: localizar una referencia `null`

Pseudocódigo

1: **procedure** INSERTAR($actual, valor$)
2: **if** $actual = null$ **then**
3: **return** $nuevo \leftarrow NodoArbol(valor)$
4: **end if**
5: **if** $valor < actual.elemento$ **then**
6: $actual.izquierdo \leftarrow$ INSERTAR($actual.izquierdo, valor$)
7: **end if**
8: **if** $valor > actual.elemento$ **then**
9: $actual.derecho \leftarrow$ INSERTAR($actual.derecho, valor$)
10: **end if**
11: **return** $actual$
12: **end procedure**

Inserción: localizar una referencia `null`



Ejemplo: insertar 65: $50 \rightarrow 70 \rightarrow 60 \rightarrow$ `null`.

Método `insertar(E elemento)`



Pseudocódigo

1: **procedure** INSERTAR($elemento$)
2: **if** $elemento = null$ **then**
3: **return** $false$
4: **end if**
5: **if** CONTIENE($elemento$) **then**
6: **return** $false$
7: **end if**
8: $raiz \leftarrow$ INSERTARRECURSIVO($raiz, elemento$)
9: $tamanio \leftarrow tamanio + 1$
10: **return** $true$
11: **end procedure**

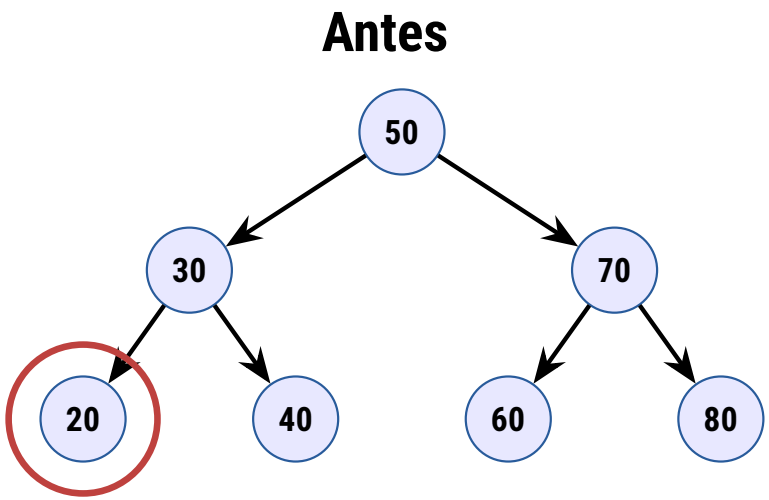
Eliminación: tres casos

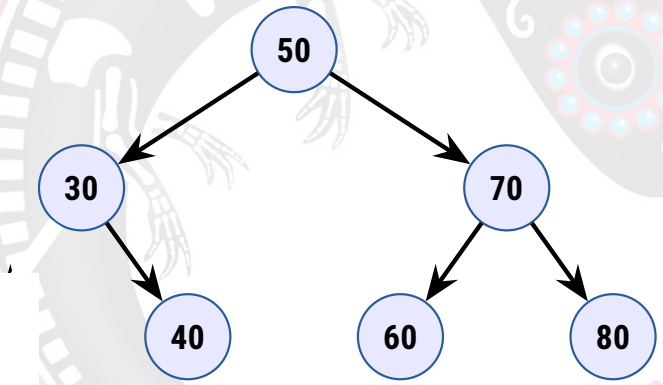
1. **Hoja:** quitar el nodo; **su padre** apunta a `null`.
2. **Un hijo:** reemplazar el nodo por su **único hijo.**
3. **Dos hijos:** sustituir su valor por el **sucesor inorden** (mínimo del subárbol derecho) y **eliminar** ese sucesor de su ubicación anterior.

¡Importante!

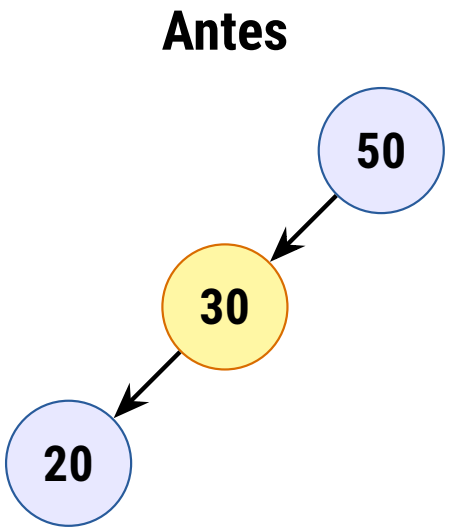
Después de eliminar, todo nodo **conserva la propiedad:** izquierda $<$ nodo $<$ derecha.

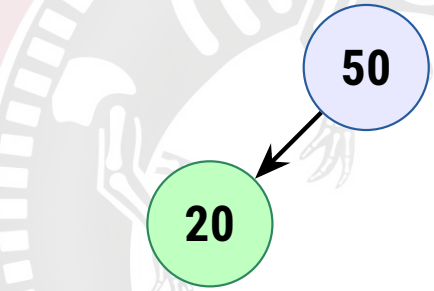
Eliminación, caso 1: nodo hoja





Eliminación, caso 2: un solo hijo



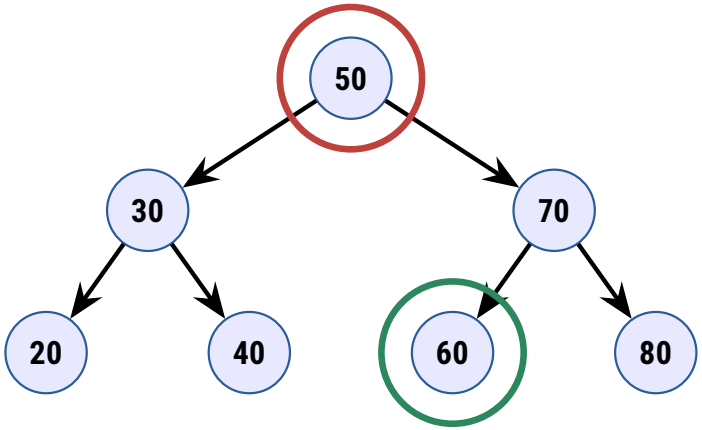


Regla

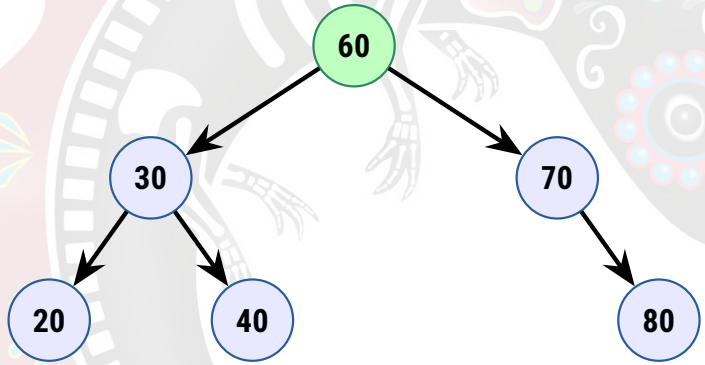
El **padre** del nodo eliminado pasa a **apuntar directamente** a su hijo. También funciona si el nodo eliminado es la **raíz.**

Eliminación, caso 3: dos hijos

Antes: eliminar 50



Después: sucesor 60



Procedimiento

Copiar 60 al nodo de 50; después eliminar el 60 original del subárbol derecho.

Eliminación: pseudocódigo recursivo I

```
1:  procedure ELIMINAR(actual, valor)
2:      if actual = null then
3:          return null
4:      end if
5:      if valor < actual.elemento then
6:          actual.izquierdo ← ELIMINAR(actual.izquierdo, valor)
7:      else if valor > actual.elemento then
8:          actual.derecho ← ELIMINAR(actual.derecho, valor)
9:      else
10:         if actual.izquierdo = null then
11:             return actual.izquierdo
12:         end if
13:         if actual.derecho = null then
14:             return actual.derecho
15:         end if
16:         sucesor ← ENCONTRARMINIMO(actual.derecho)
17:         actual.elemento ← sucesor.elemento
```

Eliminación: pseudocódigo recursivo II



18: $actual.derecho \leftarrow$ ELIMINAR$(actual.derecho, sucesor.elemento)$
19: **end if**
20: **return** $actual$
21: **end procedure**

Método `eliminar(E elemento)`



Pseudocódigo

```
1:  procedure ELIMINAR(elemento)
2:      if elemento = null then
3:          return false
4:      end if
5:      if not CONTIENE(elemento) then
6:          return false
7:      end if
8:      raiz ← ELIMINARRECURSIVO(raiz, elemento)
9:      tamanio ← tamanio − 1
10:     return true
11: end procedure
```

Encontrar máximos y mínimos

Encontrar máximo

```
1: procedure EncontrarMaximo(actual)
2:     while actual.derecho ≠ null do
3:         actual ← actual.derecho
4:     end while
5:     return actual
6: end procedure
```

Encontrar mínimo

```
1: procedure EncontrarMinimo(actual)
2:     while actual.izquierdo ≠ null do
3:         actual ← actual.izquierdo
4:     end while
5:     return actual
6: end procedure
```

Métodos `maximo()` y `minimo()`

Máximo

1: **if not** $estaVacio()$ **then**
2: **return** ENCONTRARMAXIMO($raiz$)
3: **end if**
4: **return** $null$

Mínimo

1: **if not** $estaVacio()$ **then**
2: **return** ENCONTRARMÍNIMO($raiz$)
3: **end if**
4: **return** $null$

Recorridos como métodos que devuelven `Lista<E>`

Diseño general

1: **procedure** RECORRIDO
2: $resultado \leftarrow Lista()$
3: RECORRIDORECURSIVO($raiz, resultado$)
4: **return** $resultado$
5: **end procedure**

- La lista es un **acumulador** compartido por las llamadas recursivas.
- El algoritmo no imprime datos: **construye y devuelve una lista**.
- La implementación recursiva cambia solo el lugar de `resultado.agregar(...)`.

Recorridos I

Recorrido preorden

1: **procedure** PREORDEN
2: $resultado \leftarrow Lista()$
3: PREORDENRECURSIVO($raiz, resultado$)
4: **return** $resultado$
5: **end procedure**

Recorrido inorden

1: **procedure** INORDEN
2: $resultado \leftarrow Lista()$
3: INORDENRECURSIVO($raiz, resultado$)
4: **return** $resultado$
5: **end procedure**



Recorridos II

Recorrido postorden

1: **procedure** POSTORDEN
2: $resultado \leftarrow Lista()$
3: POSTORDENRECURSIVO($raiz, resultado$)
4: **return** $resultado$
5: **end procedure**

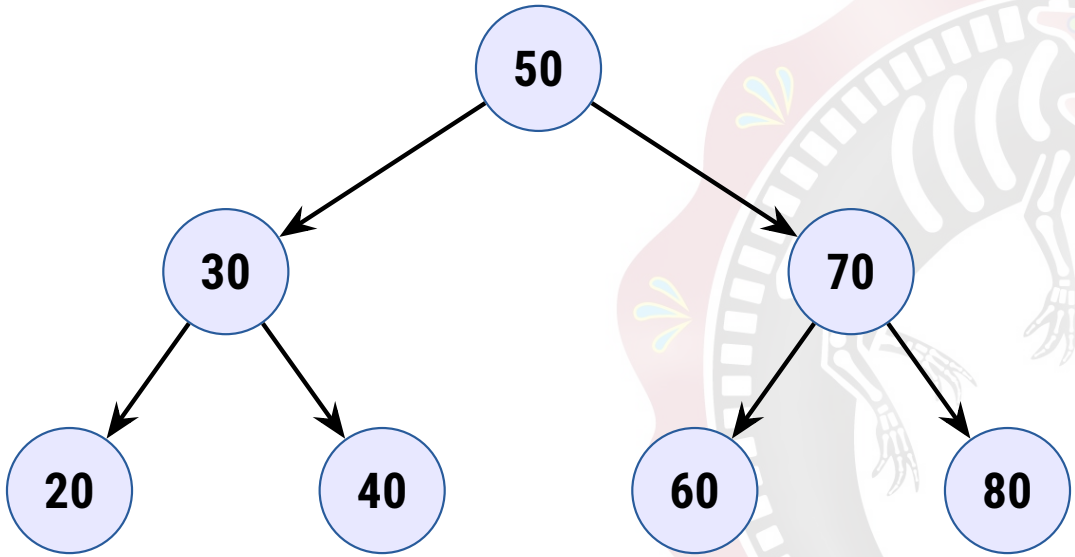


Altura del árbol: definición recursiva

Pseudocódigo

1: **procedure** ALTURA($actual$)
2: **if** $actual = null$ **then**
3: **return** -1
4: **end if**
5: $hi \leftarrow$ ALTURA($actual.izquierdo$)
6: $hd \leftarrow$ ALTURA($actual.derecho$)
7: **return** $1+$ MAX(hi, hd)
8: **end procedure**

Altura del árbol: definición recursiva

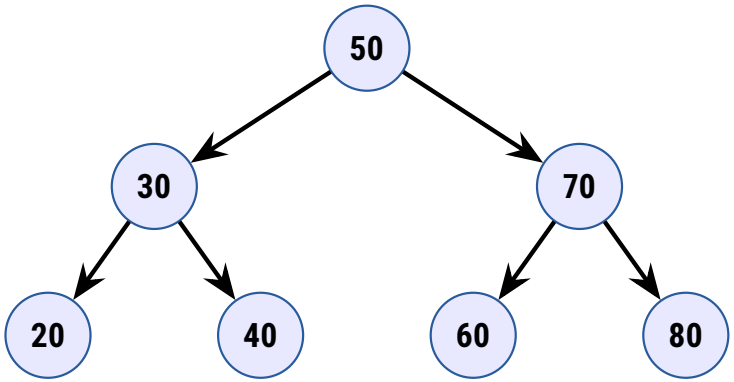


- Hojas: altura 0.
- Subárboles con raíz 30 y 70: altura 1.
- Árbol completo: altura 2.

Métodos `tamanio()` y `estaVacia()`

`tamanio()`

return $tamanio$



$\texttt{tamanio()} = 7$

`estaVacia()`

return $raiz = null$

`true` si $raiz = null$

$O(1)$

Complejidad temporal del ABB

Operación	**Altura** $O(\log n)$	**Altura** $O(n)$
Buscar	$O(\log n)$	$O(n)$
Insertar	$O(\log n)$	$O(n)$
Eliminar	$O(\log n)$	$O(n)$
Mínimo / máximo	$O(\log n)$	$O(n)$
Recorridos	$O(n)$	$O(n)$
Calcular altura	$O(n)$	$O(n)$

¡Importante!

Un ABB convencional **no se equilibra automáticamente**. Insertar datos ordenados puede producir un árbol degenerado.

Fin de la presentación



¿Preguntas o comentarios?